Arculus Recovery

User & Technical Documentation

Offline · BIP39/BIP32 · Multi-Coin · Encrypted Seed Export

Generated May 09, 2026 · ARCULUS-ARC-V2 format · MIT License

1 Overview

Arculus Recovery is a fully **offline BIP39/BIP32 seed recovery and key-derivation tool**. It is designed for users who need to inspect, validate, or recover wallet data without sending anything to a server, website, or cloud service.

The project ships in two independent but compatible forms: a single-file browser application and a Python desktop/CLI application. Both versions share the same cryptographic logic and the same encrypted seed file format.

Property	Value
Repository	https://github.com/Scrince/Arculus_Recovery
Browser app	Arculus_Recovery.html (also mirrored as index.html)
Python app	Arculus_Recovery.py
Encrypted seed ext	.arc (ARCULUS-ARC-V2 format)
Supported coins	Bitcoin, Litecoin, Dogecoin
BIP standards	BIP39, BIP32, BIP44, BIP49, BIP84, BIP86
External deps	None — zero runtime dependencies in either version
License	MIT

2 Getting Started

2.1 Hash Verification

Before using the tool, verify the SHA256 hashes of the downloaded files against the expected values published in the README. This ensures the files have not been tampered with:

```
shasum -a 256 Arculus_Recovery.html index.html Arculus_Recovery.py
```

Expected hashes (as of the current release):

File	SHA256 Hash
Arculus_Recovery.html	4298d676d6216b861e788bad94efb49c711f62cc355fc2c50aed0cd6d0e7441f
index.html	4298d676d6216b861e788bad94efb49c711f62cc355fc2c50aed0cd6d0e7441f
Arculus_Recovery.py	77938bbe0d3e2c8959d03afa28d04dbd9e3b3e78e1a70bb5716aea4e507e0f4e

2.2 HTML Version — Quick Start

Step	Action
1	Download Arculus_Recovery.html (or index.html) from the repository.
2	Disconnect from the internet.
3	Open the file directly in a modern browser — no server, no install required.
4	Enter your 12- or 24-word BIP39 mnemonic into the seed input field.
5	Select coin, script type, and derivation path. Click Derive.
6	Review addresses, keys, and validation output on-screen.
7	Optionally export results as JSON, CSV, or TXT, or encrypt the seed to a .arc file.

2.3 Python Version — Quick Start

GUI mode:

```
python Arculus_Recovery.py --gui
```

CLI mode example — derive 5 Native SegWit Bitcoin addresses:

```
python Arculus_Recovery.py \
  --mnemonic "word1 word2 ... word12" \
```

```
--derivation "m/84'/0'/0'" \
--script-type p2wpkh \
--count 5 \
--output-format txt
```

CLI output formats are **json** (default), **csv**, and **txt**.

■ Always run the tool on a trusted offline machine. Disconnect networking before opening the HTML file or running the Python script. Never share your seed phrase, private keys, or .arc backup password.

3 Features

3.1 BIP39 Mnemonic Validation

The tool performs comprehensive validation of 12-word and 24-word BIP39 English mnemonics and displays detailed results for each check:

Validation Check	Description
Word count	Confirms 12 or 24 words are present.
Wordlist validity	Verifies every word appears in the BIP39 English wordlist.
Entropy bits	Reports the entropy bit length (128-bit for 12-word, 256-bit for 24-word).
Checksum bits	Reports the number of checksum bits.
Checksum match	Validates the embedded checksum against the entropy.
BIP39 compliance	Overall pass/fail result.
Root fingerprint	Displays the BIP32 root fingerprint derived from the seed.
Keystore/seed format	Detects the likely wallet format or keystore type.
BIP39 seed (512-bit)	Shows the full PBKDF2-derived seed.
Master private key	Displays the BIP32 master private key.
Master chain code	Displays the BIP32 master chain code.
Passphrase warning	Alerts user if a BIP39 passphrase is entered (affects all derived keys).

3.2 Address & Key Derivation

After validation, the tool derives addresses and keys at the selected derivation path. Both receiving (external) and change (internal) addresses are derived:

Receiving: <account path>/0/index

Change: <account path>/1/index

3.3 Supported Script Types

Script Type	Address Format	Standard
P2PKH	Legacy (1... / L... / D...)	BIP44 — m/44'/coin'/0'

P2WPKH-P2SH	Wrapped SegWit (3... / M... / 9...)	BIP49 — m/49'/coin'/0'
P2WPKH	Native SegWit (bc1q... / ltc1q... / doge1q...)	BIP84 — m/84'/coin'/0'
P2TR	Taproot (bc1p... / ltc1p...)	BIP86 — m/86'/coin'/0'

3.4 Taproot (P2TR) Extended Support

For Taproot addresses the tool exposes additional derivation data:

Field	Description
Bech32m address	The Taproot output address encoded in Bech32m.
BIP86 purpose detection	Confirms m/86'/coin'/0' derivation path.
Taproot internal pubkey	The x-only internal public key before tweaking.
Taproot internal privkey	The corresponding internal private key.
Taproot tweak	The tweak value applied to produce the output key.
Taproot output pubkey	The final tweaked output public key.
Taproot output privkey	The corresponding output private key.
Output key parity	Even/odd parity of the output public key.

3.5 Multi-Coin Derivation Paths

Default derivation paths and address formats per coin and script type:

Coin	Script	Default Path	Address Format	Notes
Bitcoin	P2PKH	m/44'/0'/0'	1...	Legacy BIP44
Bitcoin	P2WPKH-P2SH	m/49'/0'/0'	3...	Wrapped SegWit
Bitcoin	P2WPKH	m/84'/0'/0'	bc1q...	Native SegWit
Bitcoin	P2TR	m/86'/0'/0'	bc1p...	Taproot BIP86
Litecoin	P2PKH	m/44'/2'/0'	L...	Legacy BIP44
Litecoin	P2WPKH-P2SH	m/49'/2'/0'	M...	Wrapped SegWit
Litecoin	P2WPKH	m/84'/2'/0'	ltc1q...	Default Litecoin path
Litecoin	P2TR	m/86'/2'/0'	ltc1p...	Taproot-style output
Dogecoin	P2PKH	m/44'/3'/0'	D...	Default Dogecoin path

Dogecoin	P2WPKH-P2SH	m/49'/3'/0'	9.../A...	Wallet support may vary
Dogecoin	P2WPKH	m/84'/3'/0'	doge1q...	Wallet support may vary
Dogecoin	P2TR	N/A	N/A	Disabled in the tool

Use the **Auto** script type to have the tool infer the script from the BIP44/49/84/86 purpose field when using a standard derivation path.

3.6 Export Options

Derived keys and addresses can be exported in three formats from both the HTML and Python versions:

Format	Use Case
JSON	Structured output for exact inspection or tooling. Best for programmatic processing.
CSV	Flattened row output for spreadsheet review.
TXT	Human-readable labeled sections for offline review.

■ Derived-output exports are NOT encrypted and may contain private keys and extended private keys. Treat every export as highly sensitive material. Store on encrypted media only.

3.7 Encrypted Seed Export/Import (.arc files)

The tool can encrypt and export a seed phrase to a **.arc** file for secure backup, and import it back in a later session. Imported seeds are hidden on screen by default — the user must hold the **Show Seed** button to reveal them temporarily.

Action	Behaviour
Encrypt/Export Seed	Encrypts the active mnemonic and saves it as a .arc file.
Import Seed	Loads a .arc file; the decrypted mnemonic is kept hidden in memory.
Show Seed	Press-and-hold button that temporarily reveals the imported hidden seed.

3.8 Appearance

Both the HTML and Python versions support Dark Mode and Light Mode. The HTML version persists the preference in browser localStorage. The Python version stores it as local GUI state.

4 Encryption & Security

4.1 .arc File Format — ARCULUS-ARC-V2

Encrypted seed backups are armored UTF-8 text files with an **ARCULUS-ARC-V2** header. The visible body is a single base64-encoded block rather than readable JSON, which keeps the file compact and less casually inspectable. Password-derived keys and the MAC are the true security boundary.

Example file structure:

```
ARCULUS-ARC-V2
eyJjaXBoZXIiOnsibmFtZSI6IkhNQUMtU0hBNTEyLUNUUiIsIm5vbmNlX2I2NCI6Ii4uLiJ9LCIuLi4iOiIuLi4ifQ==
```

The base64 body decodes to a metadata bundle containing:

Field	Value / Description
magic	ARCULUS-ARC
format	arculus-encrypted-seed-v2
version	2
KDF metadata	Algorithm, iterations, salt
Cipher metadata	Algorithm name, nonce
ciphertext_b64	Base64-encoded encrypted mnemonic JSON
mac_b64	HMAC-SHA512 authentication tag

4.2 Cryptographic Design

Password Normalisation

Passwords are normalized with Unicode NFKD before key derivation, ensuring consistent behaviour across different input methods and platforms.

Key Derivation

```
PBKDF2-HMAC-SHA512(password_nfkd, salt, iterations=1,000,000, dkLen=64)
  → 64-byte master key (IKM)

  encKey = HMAC-SHA512(IKM, "encryption")     // 512-bit encryption key
  macKey = HMAC-SHA512(IKM, "authentication") // 512-bit authentication key
```

Salt is 32 bytes (256 bits), generated cryptographically at random per export. Keys are domain-separated to prevent cross-context key confusion.

Encryption

```
HMAC-SHA512-CTR stream cipher:
  Nonce: 24 bytes (192 bits), randomly generated per export
  For each 64-byte block i:
    pad[i]        = HMAC-SHA512(encKey, nonce || counter_i)
    ciphertext[i] = plaintext[i] XOR pad[i]
```

Authentication (Encrypt-then-MAC)

```
MAC = HMAC-SHA512(macKey, versioned_metadata || kdf_params || nonce || ciphertext)
```

MAC verification occurs before any decryption attempt. A wrong password or any file corruption is detected immediately — no partial plaintext is ever produced.

4.3 Decrypted Plaintext Payload

Once decrypted, the payload is a small JSON object:

```
{
  "mnemonic":   "word1 word2 ... word12",
  "word_count": 12,
  "created_at": "2026-05-03T23:59:59.000Z"
}
```

Importers should ignore unknown fields for forward compatibility.

4.4 Legacy Format Compatibility

Import Format	Supported?	Notes
ARCULUS-ARC-V2 (armored)	■ Yes	Current format. All new exports use this.
JSON arculus-encrypted-seed-v2	■ Yes	Older JSON envelope with magic: ARCULUS-ARC and version: 2.
V2 with 600,000+ KDF iterations	■ Yes	Earlier V2 exports with lower iteration count remain importable.
Legacy PBKDF2-SHA256 + XOR-HMAC	■ Yes	Files without a magic header (very old exports).
arculus-encrypted-seed-python-v1	■ Yes	Legacy Python V1 format.
arculus-encrypted-seed-v1 (AES-GCM)	HTML only	Browser version only; legacy AES-GCM exports.

4.5 Threat Model

Threat	In Scope?	Notes
Offline .arc file theft	■ Protected	1M-iteration KDF makes bulk brute-force expensive. Weak passwords remain vulnerable.
File tampering / corruption	■ Protected	HMAC-SHA512 MAC detects any modification before decryption.
Network exfiltration	■ Protected	No network calls exist anywhere in either version of the tool.
Malware on the device	■ Out of scope	Client-side tools cannot protect against a compromised OS or active malware.
Screen / clipboard monitoring	■ Out of scope	Other apps and extensions may capture displayed or copied data.
Physical access	■ Out of scope	Memory forensics, screen recording, or over-the-shoulder attacks.
Weak passwords	■ Partial	KDF is intentionally slow, but guessable passwords are still vulnerable offline.
Malicious browser extensions	■ Out of scope	Extensions with page access can read in-memory JavaScript variables.

5 BIP39/BIP32 Derivation Flow

The complete derivation pipeline, from mnemonic input to final address output:

Step	Operation	Detail
1	Input normalisation	Mnemonic words are lowercased and stripped. Passphrase is NFKD-normalised if provided.
2	BIP39 validation	Word count, wordlist, entropy, and checksum are verified.
3	Seed generation	PBKDF2-HMAC-SHA512(mnemonic, 'mnemonic' + passphrase, 2048 iterations, 64 bytes) → 512-bit seed.
4	Master key derivation	BIP32 master private key and chain code derived from the 512-bit seed.
5	Account path	Child key derivation follows the selected path (e.g. m/84'/0'/0').
6	Receiving addresses	Derived at /0/0, /0/1, ... up to the requested count.
7	Change addresses	Derived at /1/0, /1/1, ... up to the requested count.
8	Script encoding	Public keys are encoded per script type: P2PKH, P2WPKH-P2SH, P2WPKH, or P2TR.

5.1 BIP39 Seed Generation Detail

Note that BIP39 seed generation (step 3) uses its own fixed PBKDF2 parameters (2048 iterations, fixed salt prefix 'mnemonic') as specified by the BIP39 standard. This is separate from and unrelated to the .arc file encryption KDF (1,000,000 iterations, random salt).

5.2 Auto Script-Type Detection

When the **Auto** script type is selected, the tool reads the BIP32 purpose field from the derivation path to infer the correct address encoding:

Purpose	Script Type	Address Format
44'	P2PKH	Legacy
49'	P2WPKH-P2SH	Wrapped SegWit
84'	P2WPKH	Native SegWit
86'	P2TR	Taproot

6 Safe Usage Guide

6.1 Recommended Offline Workflow

Step	Action
1	Download or transfer the latest project files to the target machine.
2	Verify SHA256 hashes against the values in the README (see section 2.1).
3	Disconnect all network interfaces (Wi-Fi, Ethernet) before opening the app.
4	Open the HTML file locally in a clean browser profile, or run the Python script.
5	Disable browser extensions that could read page content.
6	Perform validation and/or derivation.
7	Export only the minimum data required. Store exports on encrypted media.
8	Clear browser downloads, clipboard history, terminal history, and temp files.
9	Power down the machine when finished.

6.2 Clipboard Safety

Clipboard contents can be read by other applications, browser extensions, clipboard managers, and remote desktop tools. Avoid copying any of the following to the clipboard:

Sensitive Item	Risk
Seed phrase (mnemonic)	Full wallet compromise if intercepted.
BIP39 passphrase	Alters all derived keys; full compromise if combined with mnemonic.
Private keys / WIF keys	Direct access to funds at that address.
Extended private keys (xprv)	Derives all child private keys in the subtree.
.arc backup password	Allows decryption of the encrypted seed backup.

6.3 .arc Backup Password

✔ Use a long, unique passphrase for .arc encryption. The 1M-iteration KDF is intentionally slow, but a short or guessable password can still be brute-forced offline by anyone who obtains the .arc file.

✔ Store the .arc backup password separately from the .arc file itself — ideally in a physically secured offline location.

■ A .arc file is a backup of your seed phrase. Anyone who obtains the file AND the password has full access to the wallet. Treat .arc files with the same care as the seed phrase itself.

6.4 Python Environment

The Python version uses only the standard library — no pip packages are required. Run from a trusted Python installation. If files were transferred between machines, re-verify hashes before running.

✔ Prefer an air-gapped environment (a machine that has never been online, or a live OS booted from read-only media) for maximum security when handling high-value seeds.

7 File Reference

File / Folder	Purpose
Arculus_Recovery.html	Single-file browser application. Open directly in any modern browser.
index.html	Mirror of Arculus_Recovery.html with identical SHA256 hash.
Arculus_Recovery.py	Python version. Supports desktop GUI (--gui flag) and CLI mode.
SECURITY.md	Security policy, threat model, and vulnerability reporting process.
CONTRIBUTING.md	Guidelines for submitting issues and pull requests.
LICENSE.md	MIT License text.
Documentation/	Screenshots and supplementary documentation assets.

7.1 CLI Reference

The Python CLI accepts the following flags:

Flag	Description
--gui	Launch the desktop GUI (Tkinter).
--mnemonic	Space-separated 12- or 24-word BIP39 mnemonic (quoted).
--derivation	BIP32 derivation path (e.g. m/84'/0'/0').
--script-type	Address type: p2pkh, p2wpkh-p2sh, p2wpkh, p2tr, or auto.
--count	Number of addresses to derive (receiving and change).
--output-format	Export format: json (default), csv, or txt.

8 Reporting Vulnerabilities & License

8.1 Reporting a Security Vulnerability

Please report security issues responsibly. Do not publish exploit details publicly before maintainers have had a chance to investigate and patch.

Include in your report:

Field	Detail
Description	Clear description of the issue.
Affected file	Which file or workflow is affected.
Reproduce	Steps to reproduce the issue.
Expected/Actual	What should happen vs. what does happen.
Impact	Potential impact on users.
Fix	Suggested mitigation or fix, if you have one.

Use GitHub's private Security Advisory feature if available for the repository. Otherwise contact the maintainer through the project's preferred private channel.

8.2 License

Arculus Recovery is released under the **MIT License** — free to use, modify, and distribute, including for commercial purposes, provided the copyright notice is retained.

Source: **https://github.com/Scrince/Arculus_Recovery**